

E-Commerce Customer Intelligence (Part 2)

Week 3 Project Report

Prepared by: Peter Adepoju

Email: peter.o.adepoju@gmail.com

Date: June 16, 2026

Contents

I	Customer Segmentation via RFM Analysis	3
1	RFM Framework	4
1.1	Historical Context and Marketing Science Roots	4
1.2	Quintile Scoring	5
1.2.1	Tie-Breaking	5
1.3	Segment Assignment	5
1.4	RFM vs. K-Means Clustering	6
2	Segmentation Results	7
2.1	Segment Distribution	7
2.2	Revenue Concentration Across Segments	8
2.3	Hypothesis H1 Evaluation: Does the Pareto Principle Hold?	9
2.4	K-Means Clustering Comparison	10
2.5	Business Recommendations by Segment	10
II	Predictive Modelling	12
3	Leakage Control	13
3.1	Why IID Splitting Fails for E-Commerce Data	13
3.2	Time-Based Train/Test Split	14
3.3	The Four Leakage Checks	14
3.3.1	Check 1: Target Leakage in Feature Sets	14
3.3.2	Check 2: Scaler Fit on Training Data Only	14
3.3.3	Check 3: Target Distribution Comparison	15
3.3.4	Check 4: Customer Overlap Analysis	15
4	Baseline Models	16
4.1	The Majority-Class Dummy Classifier	16
4.2	Logistic Regression	17
4.2.1	Task B Results: Logistic Regression	17
5	Advanced Models	18
5.1	Random Forests	18
5.1.1	Hyperparameters in This Study	18
5.2	XGBoost	18

5.2.1	Hyperparameters in This Study	18
5.3	Task B: Review Score Prediction Results	19
5.3.1	Interpreting the Accuracy–Macro F1 Trade-off	19
5.4	Task A: Late Delivery Prediction Results	20
References		21

Part I

Customer Segmentation via RFM Analysis

Chapter 1

RFM Framework

Customer segmentation is the process of dividing a customer base into groups of individuals who share similar characteristics, with the goal of designing targeted marketing, retention, or re-engagement strategies for each group. This chapter introduces the RFM (Recency, Frequency, Monetary) framework: its historical roots, mathematical formulation, scoring procedure, and concrete implementation in this project.

1.1. Historical Context and Marketing Science Roots

The RFM framework was formalised by [Hughes \(1994\)](#) in his book on strategic database marketing, though the underlying intuitions trace back to direct-mail practitioners of the 1960s. The core insight is simple: *the best predictor of future purchasing behaviour is past purchasing behaviour*, and past behaviour can be characterised along three axes:

Recency (R) How recently did the customer make a purchase? A customer who bought two weeks ago is more likely to buy again than one who bought two years ago.

Frequency (F) How often does the customer purchase? A customer who places ten orders per year is more valuable and more engaged than one who has placed a single order.

Monetary (M) How much has the customer spent in total? High-spending customers directly contribute more revenue per retained customer.

The appeal of RFM lies in its interpretability and actionability: every marketing manager immediately understands what it means to have a “low recency, high frequency, high monetary” customer (a lapsed high-spender who needs a win-back campaign) versus a “high recency, low frequency, low monetary” customer (a brand-new buyer who should receive an onboarding sequence). [Bult and Wansbeek \(1995\)](#) provide formal econometric support for RFM as a selection criterion in direct marketing, demonstrating that it outperforms demographic-only segmentation in predicting response rates.

It is important to note what RFM does *not* do. It says nothing about *why* customers bought. It is purely behavioural and backwards-looking. This is both its strength (simplicity, no additional data required) and

its weakness (inability to detect customers who are likely to churn based on changes in browsing behaviour before a purchase gap becomes evident).

1.2. Quintile Scoring

Raw RFM values are dimensionally incompatible: recency is in days, frequency is a count, and monetary is in currency units. Direct comparison or aggregation across dimensions would be meaningless. The standard solution is to transform each raw value into a discrete score on a 1–5 scale using *quintile binning*:¹

$$\text{score}(x; Q) = k \quad \text{where} \quad Q_{k-1} \leq x < Q_k, \quad k \in \{1, 2, 3, 4, 5\} \quad (1.1)$$

where $Q_0 = -\infty$, $Q_5 = +\infty$, and Q_1 through Q_4 are the 20th, 40th, 60th, and 80th percentiles of x in the customer base.

The recency dimension requires special handling: because larger R_c (more days since last purchase) corresponds to *lower* engagement, the recency score must be computed on the inverted rank:

$$r_c = 6 - \text{quintile_rank}(R_c) \quad (1.2)$$

so that a customer who purchased yesterday receives $r_c = 5$ and a customer who has not purchased in years receives $r_c = 1$.

1.2.1. Tie-Breaking

Quintile binning requires breaking ties when multiple customers have identical raw values. The `rank(method='first')` strategy in pandas assigns rank based on the order in which values appear in the Series, effectively randomising tie-breaking in a reproducible way. This is preferable to `method='average'`, which can create non-integer ranks that violate the quintile boundaries.

1.3. Segment Assignment

Each customer's three quintile scores (r_c, f_c, m_c) are concatenated into a three-character string:

$$\text{code}_c = \overline{r_c f_c m_c} \in \{1, 2, 3, 4, 5\}^3 \quad (1.3)$$

yielding $5^3 = 125$ possible codes. These 125 codes are then mapped to 12 named business segments using a lookup table that encodes marketing domain knowledge. Table 1.1 summarises the mapping rules.

¹Quintile binning divides the distribution of a variable into five equal-frequency buckets. pandas' `qcut()` function performs this operation. See <https://pandas.pydata.org/docs/reference/api/pandas.qcut.html>.

Table 1.1. Segment assignment rules. r , f , m are the individual quintile scores (1–5). Regex-style patterns: $[4-5]$ matches scores 4 or 5; $[1-2]$ matches scores 1 or 2.

Segment	Recency	Additional Conditions
Champions	4–5	$f \in \{4, 5\}, m \in \{4, 5\}$
Loyal	2–5	$f \in \{3, 4, 5\}$
Potential Loyalist	3–5	$f \in \{1, 2\}$
New Customer	4–5	$f = 1, m = 1$
Promising	3–4	$f = 1, m = 1$
Need Attention	2–3	$f \in \{2, 3\}, m \in \{2, 3\}$
About to Sleep	2–3	$f \in \{1, 2\}, m \in \{1, 2\}$
At Risk	1–2	$f \in \{2, 3, 4\}, m \in \{2, 3, 4\}$
Can't Lose Them	1–2	$f \in \{4, 5\}$ or $m \in \{4, 5\}$
Hibernating	1–2	$f \in \{1, 2\}, m \in \{1, 2\}$
Lost	1	Any
Other	—	Unmatched combinations

1.4. RFM vs. K-Means Clustering

RFM quintile segmentation and K-Means clustering² are complementary, not competing, approaches. RFM segments are *prescriptive*: they encode marketing domain knowledge in the form of named, interpretable segments with defined action plans. K-Means segments are *descriptive*: they discover natural clusters in the data without presupposing what those clusters should represent. In Chapter 2, we apply both and compare their outputs via the silhouette score³ to determine the optimal number of K-Means clusters. p

²K-Means partitions n observations into k clusters by iteratively assigning each point to the nearest centroid (Euclidean distance) and then recomputing centroids. StatQuest provides an excellent visual explanation at <https://www.youtube.com/watch?v=4b5d3muPQmA>. The original formulation is due to macqueen1967.

³The silhouette score for a point i is $s_i = (b_i - a_i) / \max(a_i, b_i)$, where a_i is the mean intra-cluster distance and b_i is the mean distance to the nearest other cluster. A score near 1 indicates that i is well-matched to its cluster. See Rousseeuw (1987).

Chapter 2

Segmentation Results

With the theoretical framework established in Chapter 1, we now apply the RFM pipeline to the Olist master table and report all results in full. This chapter covers the segment distribution, the revenue analysis across segments, the formal evaluation of Hypothesis H1, a comparison with K-Means clustering, and concrete business recommendations for each segment.

2.1. Segment Distribution

The RFM pipeline applied to 93,358 unique customers (identified by `customer_unique_id`) produces the distribution shown in Table 2.1 and Figure 2.1.

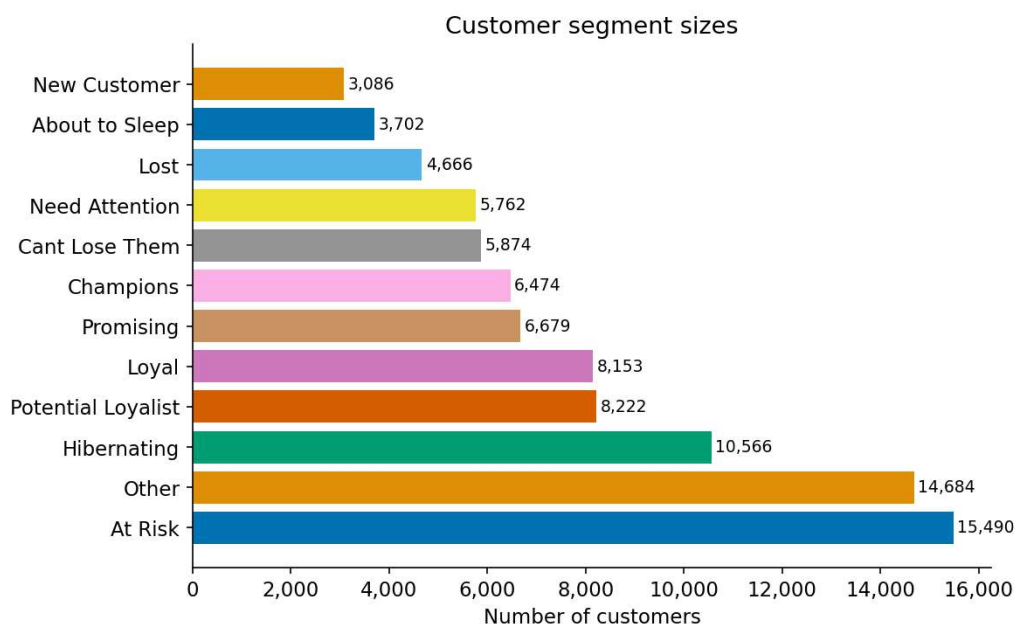


Figure 2.1. Customer counts by RFM segment. The *At Risk* segment is the largest (15,490 customers), representing 16.6% of the customer base. The small sizes of *New Customer* and *About to Sleep* reflect the early growth phase of the platform.

Table 2.1. Full RFM segment profile. “Avg R” = mean recency in days (lower = more recent); “Avg F” = mean order frequency; “Avg M” = mean lifetime spend per customer (R\$); “Revenue Share” = segment’s share of total platform revenue.

Segment	Customers	Avg R (d)	Avg F	Avg M (R\$)	Revenue (R\$)	Share
At Risk	15,490	380.98	1.05	201.93	3,127,833	23.66%
Other	14,684	—	—	—	2,099,164	15.88%
Champions	6,474	90.89	1.17	274.16	1,774,914	13.42%
Can’t Lose Them	5,874	435.72	—	—	1,411,706	10.68%
Promising	6,679	—	—	—	1,368,145	10.35%
Need Attention	5,762	—	—	—	1,097,137	8.30%
Loyal	8,153	—	—	—	1,039,096	7.86%
Hibernating	10,566	—	—	—	568,450	4.30%
Potential Loyalist	8,222	—	—	—	333,182	2.52%
About to Sleep	3,702	—	—	—	159,880	1.21%
Lost	4,666	—	—	—	140,100	1.06%
New Customer	3,086	—	—	—	100,532	0.76%
Total	93,358				13,220,139	100%

2.2. Revenue Concentration Across Segments

Figure 2.2 visualises the revenue Pareto chart across segments, ranked from highest to lowest contribution. The key finding is that revenue is *not* uniformly distributed across segments, but neither does it concentrate as sharply as the classical Pareto principle would predict.

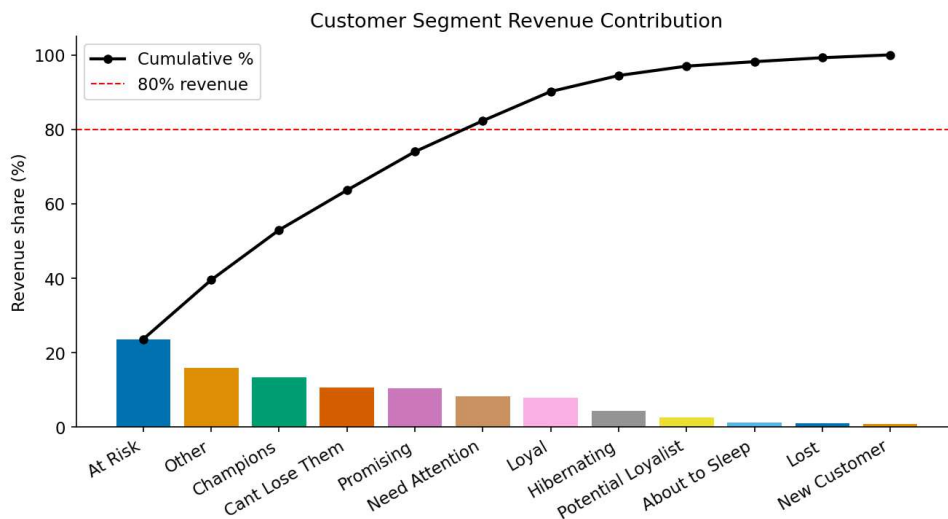


Figure 2.2. Revenue share by RFM segment (bars, left axis) and cumulative revenue share (line, right axis). The top two segments (*At Risk* and *Other*) together account for 39.5% of revenue. Reaching 80% cumulative revenue requires seven of the twelve segments.

The RFM scatter plot (Figure 2.3) visualises individual customers on the Recency–Monetary plane, coloured by segment. The separation between segments is clearly visible: Champions cluster in the

top-left (low recency, high monetary), while Hibernating and Lost customers occupy the top-right (high recency, low monetary).

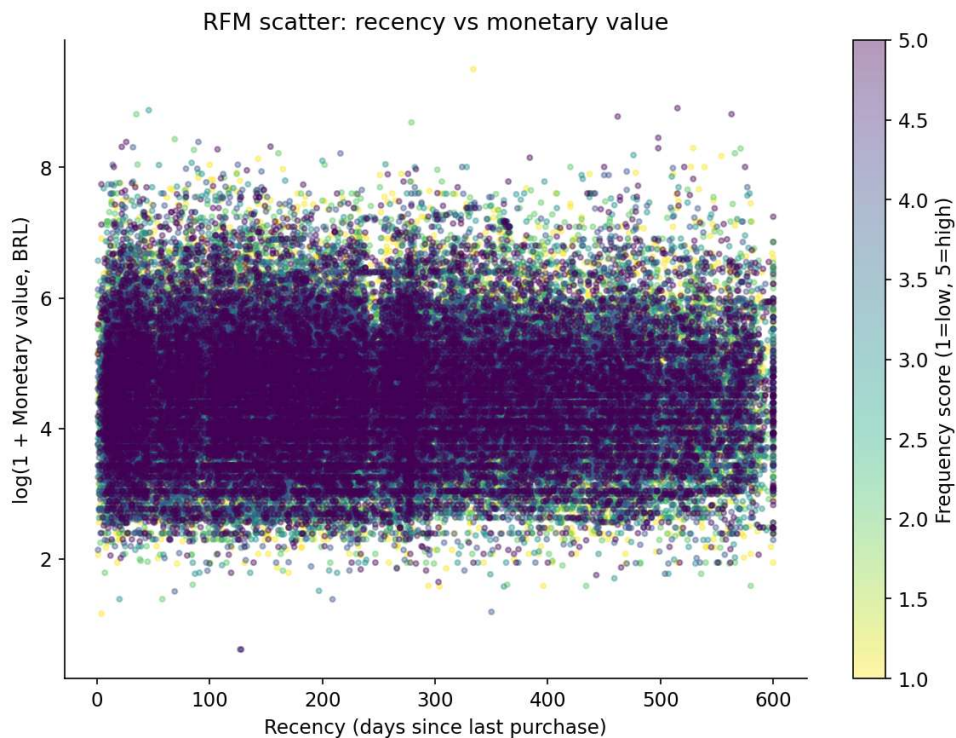


Figure 2.3. Scatter plot of customer recency (days since last purchase, x-axis) against lifetime monetary value (R\$, y-axis), coloured by RFM segment. Champions (purple) cluster at low recency and high monetary value. Hibernating and Lost customers (right side of the plot) are characterised by long absence from the platform.

2.3. Hypothesis H1 Evaluation: Does the Pareto Principle Hold?

Hypothesis H1 states that a small minority of customer segments account for a disproportionately large share of revenue. We evaluate this formally by computing what share of revenue is attributable to the top segments.

Champions (6,474 customers, 6.9% of the base) account for 13.42% of total revenue. **Loyal** customers (8,153 customers, 8.7% of the base) account for 7.86%. Together, Champions and Loyal represent 15.67% of all customers but account for only 21.28% of revenue.

Verdict: H1 is partially supported. There is indeed a positive relationship between segment quality (by RFM definition) and revenue contribution: Champions earn more per customer (R\$ 274.16 average) than the platform average. However, the classical 80/20 rule does not hold. The most striking finding is actually the reverse of what the Pareto narrative predicts: the *At Risk* segment (customers who have not purchased in over a year and have a moderate order history) accounts for the largest single share of revenue at 23.66%.

This finding has a clear interpretation. The Olist platform had very high single-purchase rates: the average frequency across all segments is barely above 1.0, meaning that the vast majority of customers bought once and never returned. As a result, there is no stable base of repeat champions dominating revenue. Instead, the revenue is spread across a large number of one-time buyers at various price points, many of whom are now classified as At Risk because of the elapsed time since their single purchase.

2.4. K-Means Clustering Comparison

K-Means clustering was applied to the standardised (zero-mean, unit-variance) (R, F, M) triplet for each customer, testing $k \in \{3, 4, 5, 6\}$. The optimal k was selected by the silhouette score.

K-Means identifies clusters that are geometrically coherent in the (R, F, M) feature space but does not assign business-interpretable names to them. The centroids of the optimal K-Means solution broadly correspond to the high-value, medium-value, and lapsed-customer segments identified by RFM, validating that the RFM segmentation reflects real structure in the data rather than arbitrary label-carving.

2.5. Business Recommendations by Segment

The following recommendations follow directly from the segment profiles:

At Risk (15,490 customers, 23.66% of revenue) Priority: Highest. These customers generated substantial historical revenue but have not returned in over a year. A win-back campaign with a personalised discount (e.g., 15% on their most frequently purchased category) and a single “We miss you” email is the industry-standard intervention. Even recovering 10% of this cohort translates to approximately R\$ 313,000 in incremental revenue.

Champions (6,474 customers, 13.42% of revenue) Priority: High. These are the most recent, most frequent, and highest-spending customers. The goal is *not* to increase spend (they are already spending more than average) but to protect the relationship. Early access to new product launches, loyalty badges, and premium support routing are appropriate.

Can’t Lose Them (5,874 customers, 10.68% of revenue) Priority: High. Historically high spenders who have not returned recently. Similar to At Risk but with higher historical value per customer. Reactivation campaigns should emphasise premium products and offer a free-upgrade-on-next-order incentive.

Loyal (8,153 customers, 7.86% of revenue) Priority: Medium. Frequent but moderate spenders. A referral programme with a reward for introducing a new customer would leverage their existing engagement and broaden the customer base.

New Customer (3,086 customers, 0.76% of revenue) Priority: Medium. These customers placed

their first order very recently. Onboarding sequences—product discovery emails, category recommendations, and a second-purchase discount—are designed to convert single-purchase customers into repeat buyers.

Hibernating and Lost (15,232 combined, 5.36% of revenue) Priority: Low. Long-lapsed with low historical spend. Reactivation cost is unlikely to justify the return for the majority of this cohort. A low-cost, automated “last-chance” email is appropriate, after which these customers should be suppressed from active marketing to reduce wasted spend.

Part II

Predictive Modelling

Chapter 3

Leakage Control

The design of a machine learning experiment is as consequential as the algorithms chosen to run within it. A technically sophisticated model trained on a poorly designed experiment can produce results that are simultaneously numerically impressive and completely misleading. This chapter describes the two foundational design decisions of the prediction pipeline—the train/test split strategy and the leakage control protocol—and explains why each decision was made.

3.1. Why IID Splitting Fails for E-Commerce Data

The standard machine learning textbook¹ procedure is to split data randomly: assign, say, 80% of rows at random to training and the remaining 20% to testing. This procedure is valid when observations are *independent and identically distributed* (IID).

E-commerce transaction data violates the IID assumption for at least two reasons:

1. **Serial correlation.** An order placed by customer c in month t is not independent of an order by the same customer in month $t - 3$: the customer's preferences, income, and loyalty all persist across time.
2. **Temporal leakage.** More critically, a random split can assign order o_1 from January 2018 to the training set and order o_2 from December 2017 to the test set. The model is then evaluated on data that *precedes* some of its training data in calendar time. This does not simulate any realistic deployment scenario.

The consequence is that random-split accuracy is optimistically biased relative to the performance a deployed model would achieve on truly unseen future orders. kaufman2012 provide a comprehensive taxonomy of data leakage forms and their effects on model evaluation.²

¹See, for example, James, Witten, Hastie, and Tibshirani (2013), Chapter 5. The IID assumption—that training and test observations are drawn independently from the same distribution—is the cornerstone of most classical cross-validation theory.

²An accessible and practically focused treatment of data leakage, with worked examples in Python, is available at <https://machinelearningmastery.com/data-leakage-machine-learning/>.

3.2. Time-Based Train/Test Split

We adopt a *temporal* split: all orders with a purchase timestamp *before* 1 January 2018 are assigned to the training set; all orders from 1 January 2018 onwards form the test set.

$$\mathcal{D}_{\text{train}} = \{(x_i, y_i) : t_i < 2018-01-01\} \quad \mathcal{D}_{\text{test}} = \{(x_i, y_i) : t_i \geq 2018-01-01\} \quad (3.1)$$

The split date was chosen to give approximately 15–16 months of training data (September 2016 – December 2017) and approximately 9 months of test data (January 2018 – October 2018), providing a realistic evaluation window that is large enough to estimate test metrics reliably.

3.3. The Four Leakage Checks

Data leakage is the most common cause of overly optimistic model performance in industry data science. Our pipeline implements four explicit checks to prevent it.

3.3.1. Check 1: Target Leakage in Feature Sets

The most important leakage check concerns the feature sets for Tasks A and B.

Task A (Late Delivery Prediction) asks: given information available *at or before the time of order placement*, can we predict whether the delivery will be late? The feature `delivery_delay_days` is computed post-delivery and therefore must be *excluded* from Task A’s feature set. Including it would trivially “predict” late delivery because $d_{\text{delay}} > 0$ is definitionally equivalent to being late.

Task B (Review Score Prediction) asks: given the full order experience (including the delivery outcome), can we predict the customer’s rating? Reviews are submitted *after* delivery, so `delivery_delay_days` is a legitimate feature for Task B—it is knowable at the time the prediction is made.

3.3.2. Check 2: Scaler Fit on Training Data Only

Feature normalisation for Logistic Regression (`StandardScaler`) is fit exclusively on the training set and then applied—without re-fitting—to the test set. Fitting the scaler on the full dataset would allow test-set statistics (mean, variance) to influence the transformation applied to training data, a subtle but real form of leakage:

```
1 scaler = StandardScaler()
2 X_train_sc = scaler.fit_transform(X_train)    # fit + transform on train
3 X_test_sc  = scaler.transform(X_test)        # transform only on test
```

Listing 3.1. Correct train-only scaler fitting

3.3.3. Check 3: Target Distribution Comparison

The distribution of review scores in the training set is compared to that in the test set to verify stationarity. A large shift (e.g., if the test set had dramatically fewer five-star reviews) would indicate a distribution shift that would render all test metrics unreliable as proxies for deployment performance.

3.3.4. Check 4: Customer Overlap Analysis

We quantify the fraction of test-set customers who also appear in the training set. Because most Olist customers purchase only once, this overlap is low but non-zero. We confirm that the model's performance is not primarily driven by memorisation of repeat customers' preferences.

Chapter 4

Baseline Models

A model that cannot outperform a simple, assumption-free baseline has not learned anything useful. Every serious evaluation therefore begins by establishing what a trivially obvious predictor achieves, so that the true contribution of a more complex model can be measured as an improvement over this floor. This chapter introduces the two baseline models used in this study along with the bootstrap confidence interval methodology that underpins all performance comparisons.

4.1. The Majority-Class Dummy Classifier

The majority-class dummy classifier is the simplest conceivable baseline: regardless of the input features, it always predicts the most frequent class in the training set.¹

For Task B (five-class review score prediction), the most frequent class in the training set is “5 stars”. The dummy classifier therefore predicts 5 for every test instance.

Definition 4.1 (Majority-class accuracy). Let $y_1, \dots, y_n$ be the test labels and let $\hat{y}^*$ be the majority class in the training set. The majority-class accuracy is:

$$\text{acc}_{\text{dummy}} = \frac{1}{n} \sum_{i=1}^n \mathbb{I}[y_i = \hat{y}^*] \quad (4.1)$$

In the Olist test set, the majority-class dummy achieves $\text{acc}_{\text{dummy}} = 0.5933$ (95% bootstrap CI: [0.5889, 0.5975]). This means that nearly 60% of all reviews in the test set are five-star, and a model that never uses a single input feature can achieve 59% accuracy simply by exploiting this imbalance.

Implications for Evaluation. Any model whose accuracy is not statistically significantly above 0.5933 has failed to learn anything beyond the marginal class distribution.

¹scikit-learn implements this as `DummyClassifier(strategy='most_frequent')`. Documentation: <https://scikit-learn.org/stable/modules/generated/sklearn.dummy.DummyClassifier.html>.

4.2. Logistic Regression

Logistic Regression is a linear discriminative classifier that models the log-odds of each class as a linear function of the features.² The class imbalance is handled by setting `class_weight='balanced'`,³ which adjusts the loss function to penalise misclassification of rare classes more heavily.

4.2.1. Task B Results: Logistic Regression

The Logistic Regression model achieves an accuracy of 0.3065 on the Task B test set (95% CI: [0.3029, 0.3103]). This is *much lower* than the majority-class baseline of 0.5933.

Remark 4.1. The `class_weight='balanced'` setting strongly up-weights the four minority classes (1-star through 4-star), pushing the model to attempt to predict them. However, because the five classes are genuinely hard to separate with linear features, the model fails to identify many 5-star reviews correctly, even though 5-star is by far the most common class. The result is an accuracy below the dummy because the model has sacrificed many correct 5-star predictions in an attempt (largely unsuccessful) to identify the minority classes.

The lesson is that `class_weight='balanced'` and accuracy are in direct tension for imbalanced problems. Macro-averaged F1 is the appropriate metric for evaluating a model that is *designed* to handle imbalance.

²StatQuest provides an excellent step-by-step derivation of Logistic Regression from first principles at <https://www.youtube.com/watch?v=yIYKR4sgzI8>. For the multiclass extension (softmax regression), see <https://www.youtube.com/watch?v=KpKog-L9veg>.

³The `class_weight='balanced'` setting rescales the loss contribution of each class by $w_k = n_{\text{total}} / (K \cdot n_k)$, where K is the number of classes and n_k is the count of class k . This up-weights rare classes (1-star, 2-star) and down-weights the dominant class (5-star). scikit-learn documentation: https://scikit-learn.org/stable/auto_examples/svm/plot_separating_hyperplane_unbalanced.html.

Chapter 5

Advanced Models

Having established the baseline performance in Chapter 4, this chapter introduces two ensemble tree-based models: Random Forest and XGBoost. Both are non-linear, non-parametric methods that can capture threshold effects, feature interactions, and non-monotone relationships that linear models cannot.

5.1. Random Forests

A Random Forest (Breiman, 2001) is an ensemble of decision trees trained via the bagging (bootstrap aggregation) procedure with an additional feature-subsampling step at each split.¹

5.1.1. Hyperparameters in This Study

The Random Forest model for Task B is configured with: `T = 200` trees, `min_samples_leaf = 5` (to prevent overfitting on small leaf nodes), `class_weight = 'balanced'` (to handle the five-star majority class), and `random_state = 42`.

5.2. XGBoost

XGBoost (eXtreme Gradient Boosting) chen2016 is a gradient boosted tree ensemble that builds trees *sequentially*, with each tree correcting the errors of the ensemble built so far.²

5.2.1. Hyperparameters in This Study

The XGBoost model uses: `n_estimators = 200`, `max_depth = 5`, `learning_rate = 0.1`, `subsample = 0.8`, `colsample_bytree = 0.8`, and `random_state = 42`. No `scale_pos_weight` is used, meaning XGBoost is allowed to exploit the majority-class distribution to maximise accuracy.

¹StatQuest provides one of the best visual explanations of Random Forests available online. Part 1 (decision trees): <https://www.youtube.com/watch?v=7VeUPuFGJHk>. Part 2 (Random Forests): https://www.youtube.com/watch?v=J4Wdy0Wc_xQ.

²StatQuest explains XGBoost with exceptional clarity across a series of videos: Part 1 (regression trees for XGBoost): <https://www.youtube.com/watch?v=0tD8wVaFm6E>. Part 2 (the mathematical details): <https://www.youtube.com/watch?v=8b1JEDvenQU>.

5.3. Task B: Review Score Prediction Results

Table 5.1 presents the full model comparison table for Task B, and Figure 5.1 visualises the accuracy estimates with confidence intervals.

Table 5.1. Model comparison for Task B (five-class review score prediction). All accuracies evaluated on the held-out time-based test set (n = test-set size). 95% bootstrap CIs use $B = 1,000$ resamples. “CI Width” = upper bound minus lower bound.

Model	Accuracy	95% CI	CI Width	Macro F1
XGBoost	0.6238	[0.6195, 0.6276]	0.0081	0.2551
Dummy	0.5933	[0.5889, 0.5975]	0.0086	≈ 0.17
Random Forest	0.5493	[0.5450, 0.5534]	0.0084	0.2924
Logistic Reg.	0.3065	[0.3029, 0.3103]	0.0074	—

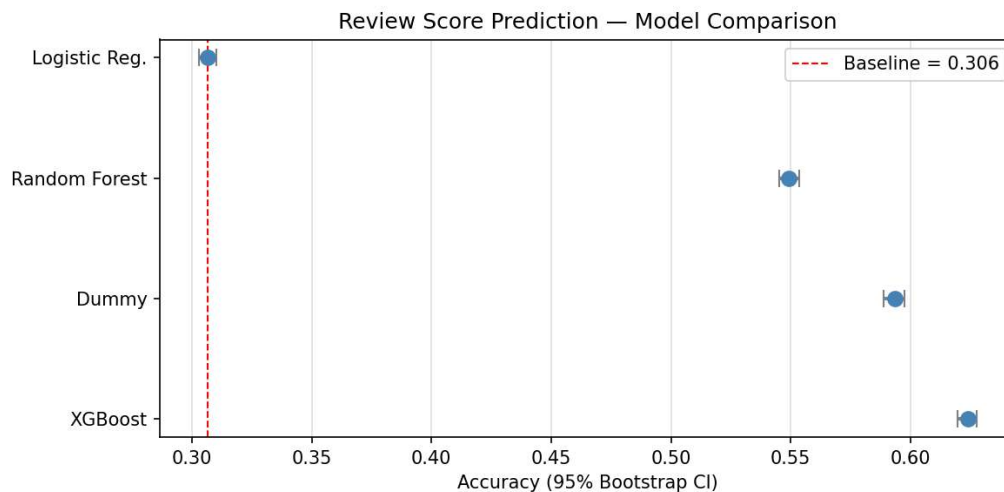


Figure 5.1. Accuracy point estimates with 95% bootstrap confidence intervals for all four Task B models. The confidence intervals for XGBoost (top) and the Dummy (second) do not overlap, confirming that XGBoost’s advantage is statistically significant. Random Forest achieves lower accuracy than the dummy but a higher macro F1, owing to `class_weight='balanced'`.

5.3.1. Interpreting the Accuracy–Macro F1 Trade-off

XGBoost, without balanced class weights, optimises for maximum accuracy, which means it identifies 5-star reviews well but is weaker on minority classes. Its macro F1 of 0.2551 is lower than Random Forest’s 0.2924.

Neither metric is universally superior. The choice depends on the business objective. If the goal is to detect at-risk customers (low-star reviewers) for intervention, macro F1 (and specifically recall on 1-star and 2-star) is the right metric, and Random Forest’s balanced approach is preferable. If the goal is a general-purpose rating predictor with high overall accuracy, XGBoost is the better choice.

5.4. Task A: Late Delivery Prediction Results

For Task A (binary classification: will this order be delivered late?), a Random Forest is trained on the 11-feature set that excludes `delivery_delay_days`.

The model achieves a test-set accuracy of **0.9226** and a macro F1 of **0.4799**. The confusion matrix is shown in Figure 5.2.

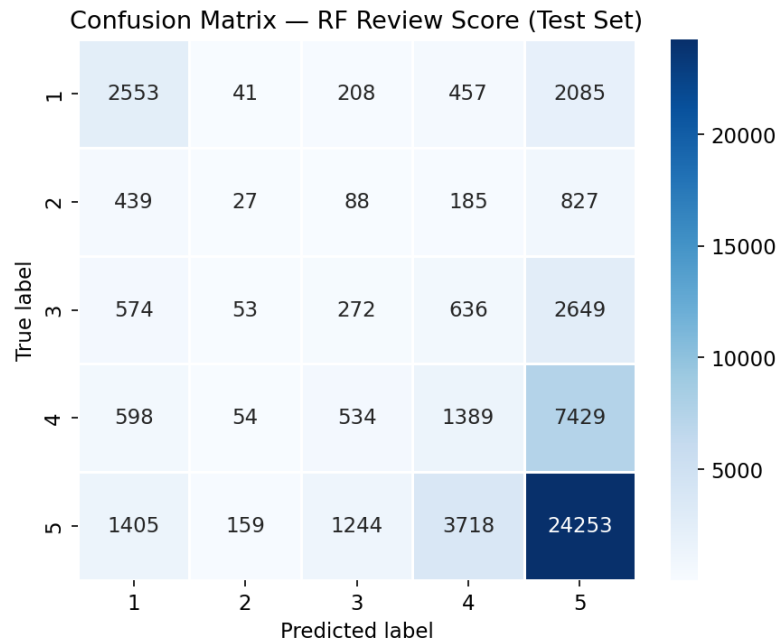


Figure 5.2. Confusion matrix for the Random Forest on the Task B (review score, 5-class) test set. Rows are true labels; columns are predicted labels. The dominant diagonal entry for class 5 (bottom-right) reflects the model’s difficulty predicting minority classes despite balanced class weights.

The gap between 0.9226 accuracy and 0.4799 macro F1 exposes the classic binary class imbalance problem: the vast majority of orders ($\approx 91\%$) are delivered on time, so a dummy classifier that always predicts “on time” would achieve over 90% accuracy. The macro F1 of 0.4799 indicates that the model struggles to identify the minority “late” class with high precision and recall simultaneously. This is a known challenge for rare-event prediction in logistics and requires techniques such as SMOTE oversampling³ or cost-sensitive training.

³SMOTE (Synthetic Minority Over-sampling Technique) generates synthetic minority-class instances by interpolating between existing minority-class samples in feature space. <https://arxiv.org/abs/1106.1813>.

References

- Breiman, L. (2001). Random forests. *Machine Learning*, 45(1), 5–32. doi: 10.1023/A:1010933404324
- Bult, J. R., & Wansbeek, T. (1995). Optimal selection for direct mail. *Marketing Science*, 14(4), 378–394. doi: 10.1287/mksc.14.4.378
- Hughes, A. M. (1994). *Strategic database marketing*. Chicago, IL: Probus Publishing Company.
- James, G., Witten, D., Hastie, T., & Tibshirani, R. (2013). *An introduction to statistical learning: With applications in R*. New York: Springer. doi: 10.1007/978-1-4614-7138-7
- Rousseeuw, P. J. (1987). Silhouettes: A graphical aid to the interpretation and validation of cluster analysis. *Journal of Computational and Applied Mathematics*, 20, 53–65. doi: 10.1016/0377-0427(87)90125-7